

Supporting Long-term Transactions in Smart Contracts

Christian Gang Liu*, Peter Bodorik*, Dawn Jutla†

*Faculty of Computer Science, Dalhousie University, Halifax, Canada
E-mail: Chris.Liu@dal.ca, Peter.Bodorik@dal.ca

†Sobey School of Business, Saint Mary's University, Halifax, Canada
E-mail: Dawn.Jutla@gmail.com

Abstract— Research on blockchains addresses multiple issues, with one being automating creation of smart contracts. One of the promising approaches, targeted for industry adoption, uses Business Process Model and Notation (BPMN) modeling to describe an application that is transformed into a smart contract. Although systems using this approach have been developed as a proof of concept, they do not address a major issue arising in the trade vertical that has applications involving collaboration of many actors over a long period of time. There are two difficulties: (i) BPMN's concept of a transaction is limited to actions of one actor, while trade transactions involve collaborations by many actors, and (ii) trade applications are long term and thus require support of transactional activities that span invocations of several methods of a smart contract. The difficulty is that a blockchain infrastructure supports the transactional properties only within the scope of one execution of one method of a smart contract. We extend the BPMN model by introducing the concept of a collaborative transaction to represent activities of many actors. In our previous research, we developed an approach, and a tool as a proof of concept, to transform a BPMN model into methods of a smart contract. In this paper we describe how collaborative transactions are supported by our approach even though the transaction activities span multiple calls to methods of a smart contract produced by transformation from a BPMN model to methods of a smart contract.

Keywords — Transactions, Blockchain, Business Processes Modeling Notation (BPMN), Discrete Event (DE) Modeling, Finite State Machine (FSM), Hierarchical State Machines (HSM), Smart Contract Methods, Atomicity, Isolation, Consistency, Durability, Privacy

I. INTRODUCTION

Blockchains have received much attention since the publication of the Bitcoin white paper in 2008 and the subsequent creation of the Bitcoin blockchain in 2009. In addition to public attention on cryptocurrency trading, business, research, and software industry communities have investigated and adopted the technology due to its desirable properties of trust, immutability, availability, and transparency. As with any new technology, blockchains and their smart contracts pose new challenges, in both the blockchain infrastructure and blockchain application development, that have slowed down the adoption of the technology.

Blockchain scalability, transaction throughput, and high cost are some of the main issues that are being addressed by research. For instance, the high cost of the consensus algorithm, which is used to ensure that a block of data to be appended to the chain is valid in face of errors or attacks, has been an impediment to adoption of blockchains. Yang et al. [1] assessed the capacity of blockchain networks and showed that it is still a critical problem and new approaches, such as those utilizing shards [1-2], are being explored for solutions.

Other issues were also tackled, issues such as the oracle problem due to the blockchain's inability to push or pull data from the external environment, privacy of recorded transactions, compatibility issues across smart contracts of different blockchains and development to smart contract. Comprehensive literature review papers on these other issues have been published with examples being [3-11].

In developing smart contracts, blockchains have complex constraints for applications, one being maintaining smart contracts in face of immutability. These constraints mean that it is even more important to use software and security engineering best practices in new approaches and methods for introducing efficiencies, as was also recommended in [7]. This reason and the attraction of ease of creation of smart contracts make Model Driven Engineering particularly promising. Literature survey in [9] reviewed existing MDE approaches to creation of smart contracts. One of the classifications offered was based on modelling techniques that included UML, BPMN, ERD, BPMN, DEMO, ArchiMate, and EMF.

Business Processes Modeling Notation (BPMN) is an standard developed by the Object Management Group® Standards Development Organization (OMG® SDO) is an international, membership-driven and not-for-profit consortium. BPMN was developed to be understandable by all business users, from business analysts, through technical developers implementing the processes, to people managing those processes. It is viewed as a de-facto standard for describing business processes. For instance, Oracle Corporation uses the Oracle Business Process Analysis Suite to model business processes using BPMN and transform a BPMN diagram into a blueprints for executable Business Process Execution Language (BPEL) processes [12]. It is for this reason that we, as other research, adopted BPMN to first model an application using BPMN modeling and then transform the BPMN model into a smart contract [15]-[19].

As others, we also developed a tool to transform a BPMN model into a smart contract. However, when we applied BPMN modeling of a long-term trade, we discovered that there are issues that arise when supporting the properties of atomicity, consistency, and isolation of long-term trade transactions.

A. Objectives

Although our overall goal is to create support for transactions for trade applications supported by blockchains in general, we start with less ambitious objectives to develop a support for transactions for applications that are expressed using a BPMN model that is transformed to a smart contract(s). More specifically, our objectives are to:

- Investigate how well the concepts of transactions are aligned in (i) trade applications (ii) BPMN modeling, and (iii) blockchains.

- Use the results of the above investigation to identify challenges in supporting transactions in trade applications when being modeled using BPMN and when transformed into blockchain smart contracts.
- Develop a mechanism for supporting transactions when smart contracts are developed by transforming a BPMN model into a smart contract(s).

The choice of starting with BPMN modeling is logical in our case as we wish to explore whether we can leverage our approach, and the supporting tool, for transforming a trade application expressed as a BPMN model into a smart contract, in achieving the above objectives. Our approach, referred to as Transforming Automatically BPMN model into Smart contract (TABS) approach [13, 15-16] and the supporting TABS [14] tool will be overviewed in the next section.

B. Contributions

- We extend BPMN modeling with a concept of a transaction that supports collaboration of a subset of actors participating in a smart contract execution.
- We incorporate the concept of an expanded BPMN transaction into the transformation process of a BPMN model into a smart contract.
- We show feasibility of our proposed approach by incorporating it into our approach and perform evaluation using our TABS tool.

This paper's contributions provide support for automated transformation of BPMN models into blockchain smart contracts with support of collaborative trade transactions. It thus moves the blockchain technology one step closer to the wide and disruptive adoption of smart contracts into in the trade domain.

C. Outline of Further Sections

Section 2 overviews the concept of transactions in contexts of trade, BPMN modeling, and blockchains. Section 3 overviews our previous work on transforming a BPMN model into a smart contract, and then it describes how to support the long-term trade transactions using our approach to transforming a BPMN model into a smart contract. Section 4 overviews the TABS tool that is augmented with the support for trade transactions as a proof of concept for feasibility of our approach. It also provides evaluation of our approach. The final section provides concluding remarks.

It should be noted that due to the space limitations, we need to omit many details, but we do strive to provide citations to references where further details can be found.

II. TRANSACTIONS IN TRADE, DBS, BLOCKCHAINS, AND BPMN MODELS

We overview the concept of transactions in the context of DBs, BPMN modeling, blockchains, and trade in order to discover how they differ and identify issues that need to be addressed in supporting trade transactions. As one can argue that the concept of transactions, as it relates to the storage, management, and manipulation of stored data, has been comprehensively developed first for DB systems, we briefly review the transactions in DB systems.

A. DB Transactions

The review of DB transactions will be brief, as the concept of DB transactions is well established and is being taught in

DB courses at any college/university level. In short, a transaction is a collection of reads and writes issued against a DB, wherein the reads and writes are delineated by the transaction begin and end commands. The DB system ensures that the transaction operations are such that properties of Atomicity (all or nothing), Consistency (correctness), Isolation (no dirty reads), and Durability are satisfied. Briefly, these properties are referred to as transactional or ACID properties. When a transaction is a collaboration of various users/participants, then each DB operation needs to identify the user who issued that operation.

B. BPMN Modeling and Transaction Subprocess

BPMN has four basic categories of elements: swim lanes, which are referred to in BPMN documentation as swimlanes, flow elements, data, and artifacts plus two additional categories [20]. We briefly describe each category and then we discuss BPMN transactions.

Swimlane: It represents a participant in a business process.

There are two types of swimlanes: Lanes and Pools. A pool with lanes represents an organization while lanes within a pool represent individual participants or organizational units, such as departments, within that organization. Only message flows can cross pools - no other BPMN elements can do so.

Flow Elements: They are used to form a process flow and they include: *Task* and *Sub-process*, which are referred to as *activities*, *events* and gateways that are used for flow control. These elements are connected either by sequence flows or by message flows. A task represents an atomic activity that is not broken down any further. A sub-process is used to represent complex work that is further broken down into activities represented by a Business Process Diagram (BPD) using BPMN. Any level of nesting may be obtained. The restriction is that *BPMN elements within a sub-process may be connected only to other elements within the same sub-process* and are thus localized to the sub-process that contains them. Another restriction is that a *subprocess has exactly one incoming and one outgoing sequence flow*.

Events: There are three types of events: Start, Intermediate and End. Start and End events are special in that they denote the start and an end of an activity or a business process. Intermediate events include many different types with examples, such as a message timer or escalation.

Gateways: They are used to represent the flow of a business process. A gateway may have one or more incoming flows and one or more outgoing flows to represent merging and splitting of flows, respectively.

Data Objects: They are used to show data that is input into or produced by a flow element, in which case Data-input and Data-output icons are used. There is also a Data store element.

Associations and Data Associations: They are used to provide any additional details and may be attached to various BPMN elements.

There are many details that have been omitted; however, a simple diagram, shown in Fig. 1, should exemplify the basic concepts. The figure shows three actors in three pools. Customer actor prepares an order and passes it to the Order-handler actor who, checks the order and then concurrently sends a confirmation of the order to the Customer actor and sends to the Shipper a request to schedule a shipment. Once

Customer receives from Shipper the confirmation that the shipment was scheduled, the order is accepted.

OMG BPMN 2.0 specification [20] enables modeler to define a transaction on a sub-process with visual representation being that BPMN sub-process icon has a double-line boundary to indicate that the sub-process should be atomic, i.e., it should have the all-or-nothing property: Either all of the activities represented by the sub-process complete successfully or an exception is raised that needs to be serviced by compensation activities. Recall also that the BPMN subprocess icon has exactly one incoming and one outgoing sequence flow.

C. Blockchain Transaction

Documentation on the various blockchains generally states that a transaction is any change to the state of the application, which is stored on the blockchain.

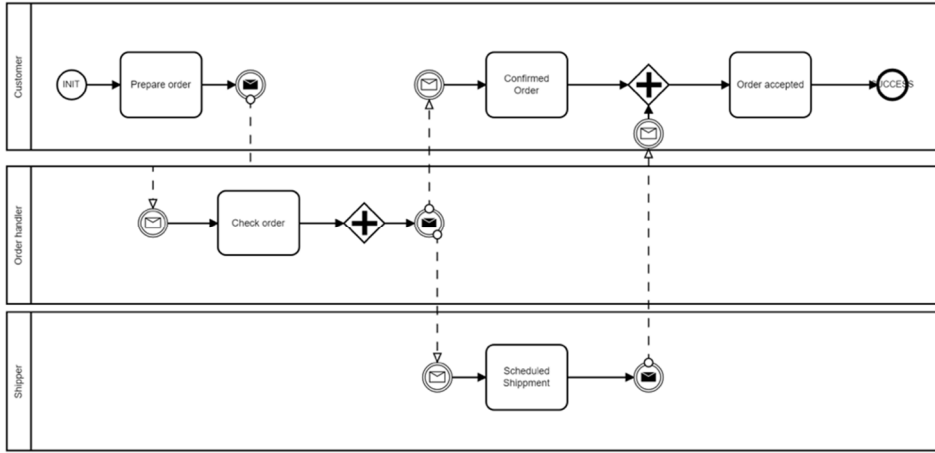


Fig. 1. BPMN diagram

Definition: A blockchain transaction is a collection of updates to the ledger state made by execution of one smart contract method.

In a blockchain, updates made by an execution of a smart contract method are submitted for inclusion in a block that is appended to the blockchain. Block verification ensures that updates made by a smart contract method are consistent relative to the state of the ledger thus far and relative to all transactions within a block. Consequently, writes of a smart contract method are *atomic* (either all or none of the writes are applied to the blockchain), *consistent* and *isolated* in that no methods of other smart contracts may access the transaction's updates until their block is appended to the blockchain. As updates are written to the blockchain, which is replicated, they are *durable* in that they are not lost after the transaction successfully completes.

D. Trade Transaction

As described in the introduction, trade transactions are long-term, involving seconds, minutes, hours or even days, and may involve collaboration of many participants to achieve a common goal. During collaboration, a group of participants that is a subset of participants in a trade activity, may perform collaboration pertaining to that group only. Furthermore, that group's collaboration should be *atomic*, either all activities must be completed successfully, or the result must be the same as if none of the collaboration activities were performed.

III. SUPPORTING BPMN COLLABORATIVE TRANSACTIONS

Recall from the previous section that a blockchain natively supports the properties of consistency, durability, and isolation, however, with some caveats that need to be discussed.

Atomicity, Consistency, and Isolation: As was discussed already in the previous section, blockchain supports these properties, but only for updates of one contract method execution. Thus, if transaction activities include execution of more than one smart contract method, these properties are NOT supported.

Durability: Durability is supported by blockchain for all updates if they appear in a block already appended to the chain.

Thus, the transaction support needs to be provided to support the properties of Atomicity, Consistency and Isolation for applications that have transactions that involve execution of more than one smart contract method, which occurs frequently in trade transactions as they are long-term and involve collaborations of actors.

However, the first issue we addressed is due to modeling of long-term collaborative trade transaction in BPMN – there is no suitable modeling concept in

BPMN for such a transaction. To address this issue, we introduce the concept of a *BPMN collaborative transaction*, including its semantics and properties that they should have and that need to be supported by a system. We then overview our approach to transformation of a BPMN model into a smart contract, which is referred to as **TABS** that is short for **Transforming Automatically BPMN model into Smart contract** and then describe how the TABS approach supports a long-term trade transaction represented in BPMN as a collaborative transaction.

A. BPMN Collaborative Transaction

The BPMN specification [20] restricts a transaction to the scope of a subprocess that is limited to a swimlane. Hence, a sub-process must be executed locally by one actor, while a trade transaction needs to support collaboration of a number of actors. For that reason, we extend the BPMN concept of a transaction sub-process to cross swimlanes.

Another issue is that in BPMN, a transaction process boundary is delineated by an actor entering the activity via the incoming sequence flow into the subprocess, while that transactional activity ends upon exit from the subprocess via the outgoing sequence flow. However, for trade transactions, the subprocess is a collaboration of actors/participants, and hence we need to allow the participating actors to enter the subprocess and perform their part in collaboration. The subprocess itself must keep track of the state of the collaboration in order to determine conditions for the start and end of a transactions.

As a consequence of the above, we define a transactional BPMN collaboration in the following way:

Definition: A BPMN *collaborative transaction* is a compound activity that has the following properties:

- It has one incoming and one outgoing sequence flow and it is in its own pool.
- The semantics of the scope of the transactional activities are defined as follows:
 - A collaborative transaction has a status, called *transaction-status*, with possible values: *no-transaction* or *transaction-active*, with the initial value being *no-transaction*
 - When a control flow arrives via the incoming sequence flow, if the *transaction-status* is *no-transaction*, then the *transaction-status* is set to *transaction-active*.
 - Transaction is in an active state until control flow exits the collaboration and the *transaction-status* is *no-transaction*.
- *Atomicity* (all-or-nothing property) – any updates to the ledger by a transaction must be atomic.
- *Consistency* – a collaborative transaction transforms the state of computation from one consistent state into another consistent state.
- *Privacy and Isolation*: Any activities within a collaborative transaction that affects the state of the ledger need to be private, that is visible only to the actors who participate in the collaboration. However, once the transaction completes, the ledger must be updated atomically.
- *Durability*: Once a transaction completes, its ledger updates must be lasting (not lost).

However, as per BPMN specification for a sub-process, a collaborative transaction, has only one incoming and one outgoing sequence flow for the same reason as a transaction subprocess: An atomic activity needs to be localized as otherwise atomicity, isolation, and consistency may not be enforceable.

To summarize, a transaction mechanism to support needs to ensure the atomicity, consistency, isolation, and privacy properties for a collaborative transaction that involves invocation of more than one smart contract method.

B. TABS Overview

Creating a smart contract from a BPMN model has been addressed by a number of researchers [17-19] and produced Lorikeet [19] and Caterpillar [18] projects as proofs of concepts. We also addressed that problem [15-16] but departed from the other work in major aspects that we plan on to exploit in supporting collaborative transactions. Here we overview the TABS approach to provide background before we describe how TABS supports trade transactions. For more information, please see [13] and [14].

The TABS approach, although similar to work of others in that it starts with a BPMN model and produces the methods of smart contract, is differentiated from the previous work in the following aspects:

1. We first transform the BPMN model into an equivalent representation using a DE-HSM multi-modal model [19], in which concurrency is represented using Discrete Events modeling and functionality is modeled using FSMs.

2. We support automatic transformation of the DE-HSM model into the methods of a smart contract(s). Choreography, that is scheduling of sequencing of execution of individual tasks in the collaboration, is represented by a DE-HSM model consisting of interconnection of DE-FSM sub-models.
3. Each individual DE-FSM sub-model is an independent component, also referred to as a pattern, with a desirable property that once the pattern execution starts, it is localized until execution leaves that pattern. TABS utilizes this property in facilitating off-chain processing of such patterns. Off-chain processing is a concept in which the methods of the smart contract are divided into two smart contracts, one execute on the mainchain and the other on a separate blockchain, referred to as a sidechain wherein the mainchain methods invoke the methods on the sidechain and thus control overall execution while sidechain methods provide the supporting computation. The concept was originally proposed for reducing the cost of processing, by moving computation to a cheaper sidechain.
4. Modeler is provided information on the patterns/sub-models and selects those sub-models which should be processed off-chain. The TABS system transforms the DE-HSM model into smart contracts, one deployed and executed on the mainchain and the other smart contract deployed and executed on a sidechain. The mainchain contract invokes the methods of the sidechain contract that implements the patterns that were selected by the developer to be executed on a sidechain. The mainchain may be either one of Ethereum (private or public) and Hyperledger fabric, while sidechain may be one of Quorum (Ethereum-EVM sidechain) or Hyperledger fabric channel.

The selection of which sub-models should be executed off-chain, i.e., on a sidechain, is for the purposes of reducing the computational cost of processing on the mainchain. To perform an informed selection of which models are to be executed on a sidechain, the developer is provided information on the cost of executing the sub-model on the mainchain vs. being executed on a sidechain, in which case overhead costs due to execution on a sidechain are also provided.

For simplicity in this early work on supporting transactions, we made a number of simplifying assumptions that are described in detail in [16] of which major ones are that there are no loops in the BPMN model, no support for transaction subprocess, and no support for looping and concurrent types of subprocesses.

C. TABS: Supporting Collaborative Transactions

One of the first issues that needs to be addressed is BPMN modeling of applications that have collaborative transactions – we briefly discuss why we chose an approach based on separations of concerns. Following this, we describe how TABS supports collaborative transactions.

1) Using Separation of Concerns in BPMN Modeling

In a previous section we defined a BPMN icon, to represent a collaborative transaction, and its semantics. However, without any additional assistance provided to the modeler when creating the BPMN model, the modeler is

forced to think right from the start about how to include the collaborative transaction within a BPMN model. That a collaborative transaction has properties, such as atomicity and privacy, which are non-functional properties, interferes in the BPMN modeling process that concentrates on the representation of how software functions using control flow notation. To alleviate this problem, instead of the modeler starting with the collaborative transaction representation and then “modeling the rest of the collaboration” around the collaborative transaction, we use the approach based on separation of concerns between the functional representation and the non-functional transactional properties: The modeler first creates the BPMN model, and after the model is completed, the TABS system provides to the modeler patterns that may be selected to represent the collaborative transactions:

1. The modeler first develops the BPMN model.
2. To define a transaction collaboration, the system first provides the modeler with information on all patterns in the BPMN model that have only one incoming sequence flow and one outgoing sequence flow.
3. From the selected patterns, the modeler selects those that should be supported as transaction collaborations.

To identify the BPMN patterns that can serve as transactional collaborations, the TABS system represents a BPMN model as a Directed Access Graph (DAG), in which nodes are the BPMN icons and sequence flows are represented by edges, and uses an algorithm to find all subgraphs in the BPMN graph that have only one incoming edge (incoming sequence flow) and one outgoing edge (outgoing sequence flow)[16:16].

For instance, consider the BPMN model shown in Fig. 1 for the Order use case. Once the BPMN model is created, the algorithm to find all patterns (subgraphs of the BPMN graph that satisfy the constraint that there is only one incoming and one outgoing sequence flow from the subgraph), the system uses the algorithm to find all subgraphs that have exactly one incoming and one outgoing sequence flows and show them to the modeler who then selects which ones should be collaborative transactions. Fig. 2 shows found subgraphs S1, S2, and S3, which are shown to the modeler to select those that should be collaborative transactions.

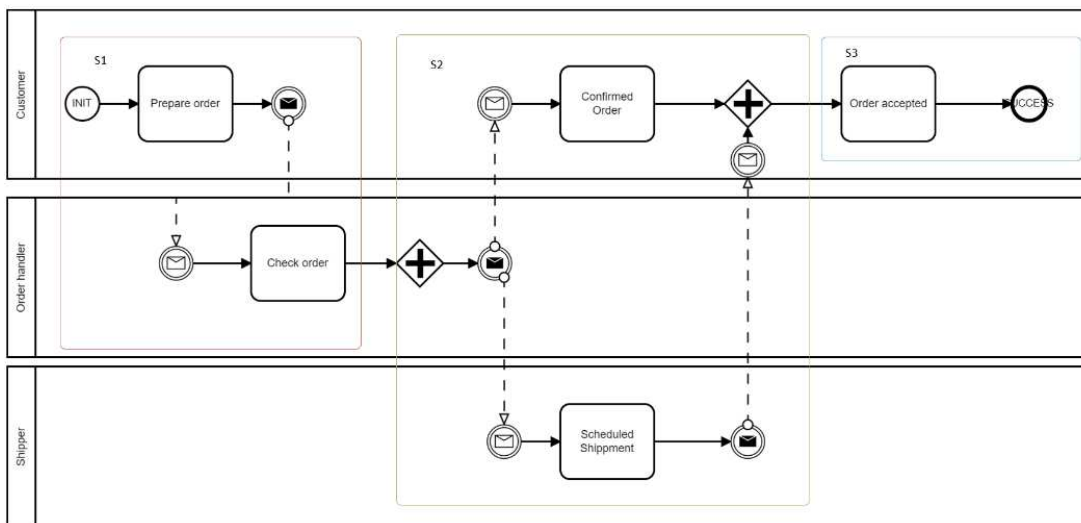


Fig. 2. BPMN diagram with found subgraphs

Please note that the figure does not show all found subgraphs as the figure would get too “busy”. For instance, a union of subgraphs S1 and S2 (or the union of S2 and S3) forms a subgraph with one incoming and one outgoing sequence flow. Furthermore, the INIT and SUCCESS events stand for the START and END events, respectively.

BPMN models a transaction subprocess as a single icon with one incoming and one outgoing sequence flow for a fundamental reason – it provides for isolation of visibility (no-dirty reads) to the state of an incomplete transaction activity. The collaborative transaction definition also has the same constraint, in that the collaborative transaction has only one incoming and one outgoing sequence flow in order to enable clear definition of the scope of the transaction, i.e., it is used to states of execution that correspond to the beginning and ending of the transactional activity. If many incoming and outgoing sequence flows were allowed, the transaction scope identification would not be intuitive and would be far more complex for enforcement of atomicity, isolation, and consistency properties, if possible at all.

The modeler chooses, from the presented patterns, those patterns that require the collaborative transactional properties – once the smart contract is deployed, the system ensures that the chosen collaborative pattern do possess the transactional properties. And how these properties are enforced by the system is described in the next subsection.

We should note that, although we described a new BPMN icon in a previous section to represent a collaborative transaction, we did not modify the BPMN diagram editor to include it. If our suggested collaborative icon were included in BPMN diagram editor, once the modeler identifies a pattern that should exhibit properties of the collaborative transaction, the selected pattern in the BPMN diagram would be replaced by the collaborative transaction icon in a manner similar to the transaction subprocess representing a collapsed pattern.

2) Supporting Collaborative Transactions

We now describe how we use the TABS approach, and the TABS tool we built as a proof of concept, to enforce the properties of a collaborative transaction. Recall that in the TABS approach, the BPMN model is first transformed into the DE-HSM model that is a collection of DE-FSM sub-models that are interconnected together. The sub-models are BPMN patterns, wherein each sub-model has one incoming and one outgoing sequence flow. DE modeling is used to represent the concurrency aspects, while each sub-model’s functionality is represented by an FSM or a set of concurrent FSMs. Interconnections between the sub-models represents the control flow amongst the sub-models, while once execution in a sub-model starts, its execution by

control arriving on an incoming sequence flow, is functionally independent of the other sub-models.

Recall that the issue is that the system must provide for the transactional properties of atomicity, isolation, and privacy in face of a collaborative transaction activity spanning multiple calls to the smart contract method – a blockchain infrastructure supports these transactional properties only for execution of one smart contract method and not across several methods. However, also recall that the TABS system supports execution of patterns, which exhibit certain property, on a sidechain for the purposes of reducing the cost of processing (on the mainchain by executing selected patterns on a sidechain). The special property that a pattern needs to exhibit, so that it can be processed on a sidechain, is that the pattern, when represented as a BPMN graph, has only one incoming sequence flow to the pattern (from nodes that are external to the pattern) and one outgoing sequence flow to BPMN graph nodes that are external to the pattern. However, this is the same property that a BPMN collaborative transaction has! This means that the patterns that are identified by the TABS systems, as suitable for processing on a sidechain, are also the same patterns that satisfy the definition of a collaborative transaction as they are both represented by subgraphs that have the same property of only one incoming and one outgoing sequence flows.

Thus, the functionality provided by the TABS approach and its tool to process selected patterns for processing on a sidechain, can also be used to identify the patterns that can be used as collaborative transactions. Once a collaborative pattern is identified, its transaction properties are enforced off-chain: Once a pattern is selected by a modeler, the pattern is deployed on a sidechain where it is executed, wherein the writes performed on the sidechain are “migrated” to the mainchain only *after completing the pattern’s execution by the control flow leaving that pattern via the outgoing sequence flow and the collaborative transaction status is no-transaction*. Consequently:

- *Isolation (no dirty reads)* is supported as all writes are performed on a sidechain first and are recorded on the mainchain only after the pattern completes.
- *Privacy* is also supported as only actors who participate in the collaboration represented by the pattern enter the pattern. Actors who do participate in the smart contract execution but not in the collaboration, do not have visibility into the collaborative process while it is being executed on a sidechain. Of courses, all participants in the smart contract have access to the results of the collaboration recorded on the blockchain as any writes to the blockchain ledger made on a sidechain are migrated (written/recorded) on the mainchain.
- *Atomicity* is supported as the writes made on the sidechain are propagated to the mainchain only when the execution control leaves the pattern and the status of the

collaborative transaction is set to *no-transaction*, which occurs during execution of one smart contract method only. Thus, all writes recorded on the sidechain are recorded on the mainchain during one smart contract method execution, which means that the property of *atomicity* is also enforced.

IV. EVALUATION USING TABS TOOL

We developed a tool, as a proof of concept of our TABS approach, and we used it to evaluate the support of collaborative transactions using the supply-chain use. The BPMN model for the supply-chain use case is shown for Fig. 3.

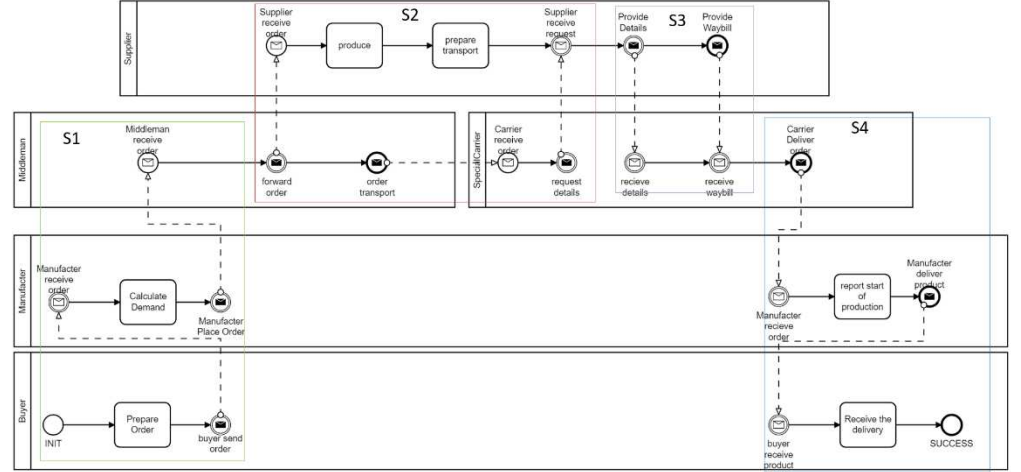


Fig. 3. BPMN model for the supply-chain use case

We chose the supply-chain use case as it has been used in several research reports on creating smart contracts for a BPMN model, e.g., [22]. The use case begins with Buyer issuing a new order to Manufacturer. Once Manufacturer receives the order, she/he calculates the demand and places an order with Middleman. Middleman concurrently sends the order to Supplier and orders transport from Carrier. Supplier fabricates the product and prepares it for transport. Carrier, upon receiving the request from Middleman, requests details from Supplier. Supplier provides the details to Carrier who then prepares a waybill and sends it to carrier. Upon receiving details about the product and the waybill, both from Supplier, Carrier delivers the order to the manufacturer. Upon receiving the order, Manufacturer starts the production and when that is finished, it delivers the product to Buyer, who receives the order. The figure also shows four faint rectangles that were not part of the BPMN diagram produced by the Camunda BPMN diagram editor, but were added by us to delineate four subgraphs, S1, S2, S3, and S4 that were found by the TABS tool as candidate collaborative transactions that will be discussed later.

Fig. 4 shows a screenshot of the TABS tool when creating the BPMN model of the supply-chain application. The TABS tool invokes Camunda software platform that is used by the developer to create a BPMN model in graphical form and then store it in XML format. The first two tabs deal with BPMN modeling using the Camunda platform. Once the BPMN model is developed and stored in an XML format, the modeler’s interaction with the tool consists of (i) guiding the transformation of the BPMN model (in XML format) into a smart contract using the tabs/buttons and (ii) supplying template methods for BPMN tasks with code.

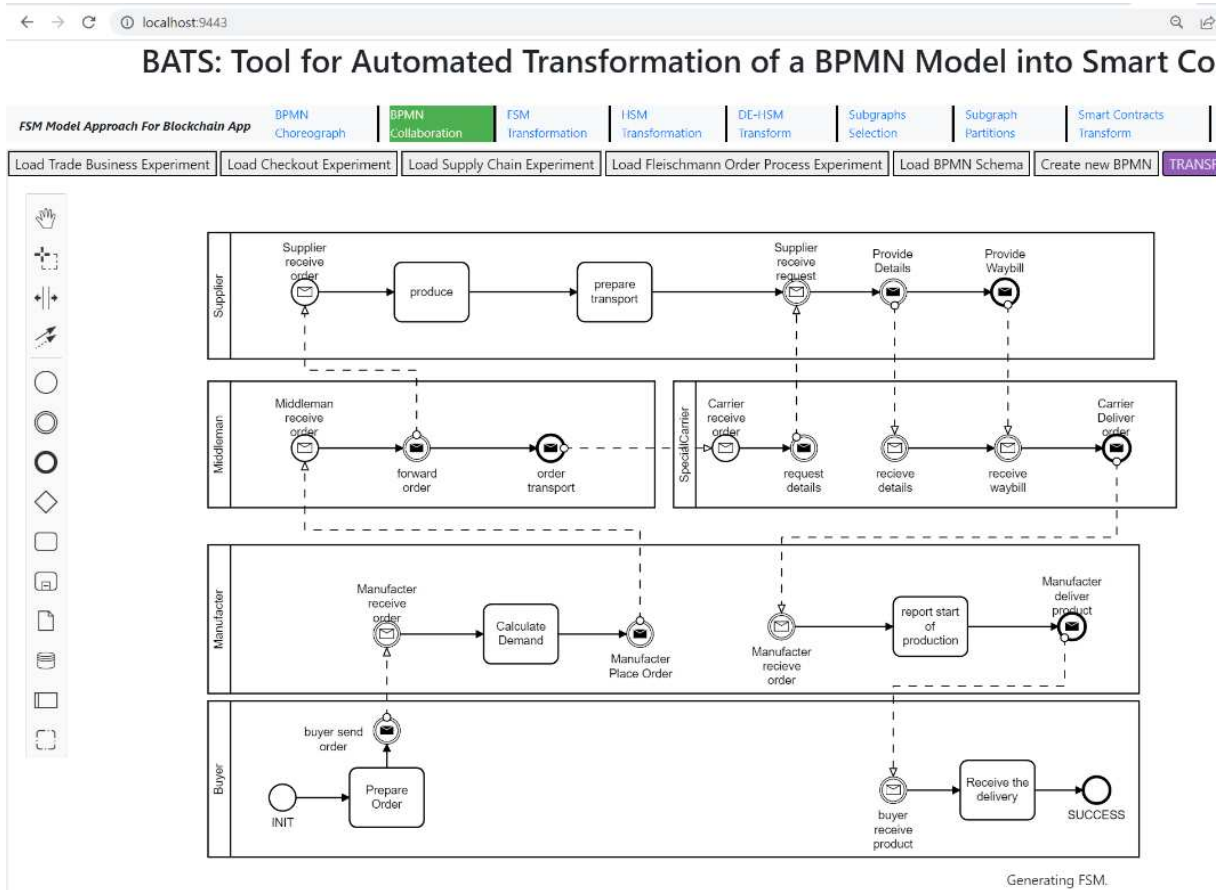


Fig. 4. Screen shot showing TABS tool with created BPMN diagram

The TABS tool does not have a specification for the script/code to be executed by each task and hence the developer needs to supply it. However, each task is independent from the rest of the system as it only access to the data stored on blockchain and its input and parameters.

We facilitate creation of a smart contract and sidechain smart contract(s) either on Ethereum or Hyperledger fabric blockchains. The tool currently supports using two blockchain types: (i) Hyperledger fabric and (ii) Ethereum or EVM-based blockchain such as Quorum. Either type can be used for the mainchain or for a sidechain(s), wherein the mainchain smart contract invokes methods of the smart contract deployed on a sidechain.

Fig. 5 shows a screenshot after the supply-chain BPMN model was transformed into the DE-HSM model having four subgraphs that were found by the tool as having one incoming and one outgoing sequence flow. The subgraphs are identified by the TABS tool (for the BPMN graph representing the supply chain use case). The subgraphs S1, S2, S3, and S4 correspond to the subgraphs of the same name shown in Fig. 3.

The modeler then selects which of the subgraphs are to be processed on a side chain as collaborative transactions and selects which of the blockchains is to be used as a mainchain and a sidechain. The tool currently supports the following blockchains: Public Ethereum Mainnet, Quorum, Hyperledger, Hyperledger Microfab, POA, xDai, Ropsten, Rinkeby and Ganache testing environment.

Following the selection of the chains to be used as the main and as a sidechain(s), the modeler then guides (that is

chooses tabs on the screen) the tool to deploy the contracts on the main and sidechain. Similarly, the modeler guides the model's execution either in a step-by-step mode or in executing the full contract. Input to the application is synthetic, prepared by the tool and being available immediately to the executing application. We measured delays for executing on the mainchain only, without selecting any collaborative transaction. We also specified deployment when the modeler chose the subgraph S2, which is shown in Fig. 6, to be deployed on a sidechain as a collaborative transaction

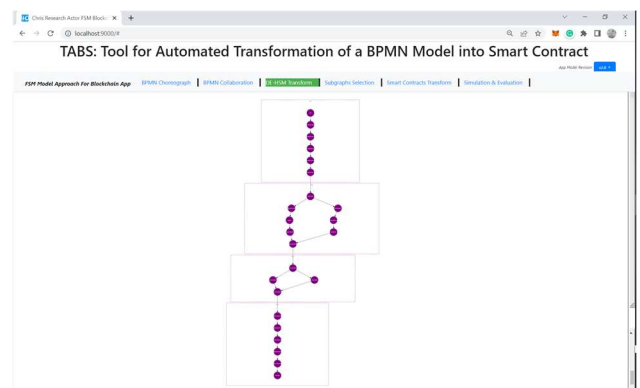


Fig. 5. TABS tool screenshot showing candidate collaborative transactions.

Table 1. Average delays with and without collaborative transaction

	Without X-action	With X-action
Delay	5.54 ~ 6.91 sec	0.53 ~ 2.84 sec

Table 1 shows observed delays for executing one contract method, over all contract methods executed, with and without

collaborative transactions executed on a sidechain.

For the main chain, the Ropsten and Rinkeby public test chains were used in an alternating fashion, but with the same parameters. For the sidechain, two public test chains POA and xDai were used, and two permissioned chains, Quorum and Hyperledger Microfab, all used with the same or similar parameters.

The major component of the delay was due to validation and the consensus algorithm. Secondly, the variability in delay was due to the variations in the communication delays. Delays in executing on the two mainchains, Ropsten and Rinkeby were not only comparable, but quite close.

Delays due to processing on a sidechain were considerably lower than those due to processing on the mainchains. This is not surprising as Quorum, POA, and xDai, which we used as sidechains, use consensus mechanisms whose delays are much less than those of the mainchains, for which we used Ropsten and Rinkeby chains that use the proof of work algorithm for consensus. Hyperledger Microfab, being a permissioned chain, uses a different mechanism to achieve consensus that is not computationally demanding: Transactions are endorsed as they are submitted and when block is added, its transactions must be organized in a serializable order to ensure consistency (correctness, no double spending).

The fact that processing on a sidechain is used takes much less time deserves comments. Although a transaction submitted by a smart contract method executed on a sidechain is stored in a block that must be validated and approved by a consensus algorithm, if the sidechain delays and costs are much less, then they provide incentives for moving computation from an expensive mainchain to a sidechain. In fact, the reduction in delays due to processing on less expensive sidechains more than outweighs the delays incurred due to processing on a sidechain.

And, because processing on a less expensive sidechain is so attractive, that also allays any concerns that using collaborative transactions would increase delays too much.

V. RELATED WORK

To the best of our knowledge, there is no report on work in supporting transactions for blockchain applications when a transaction's activities are performed by more than one method of a smart contract.

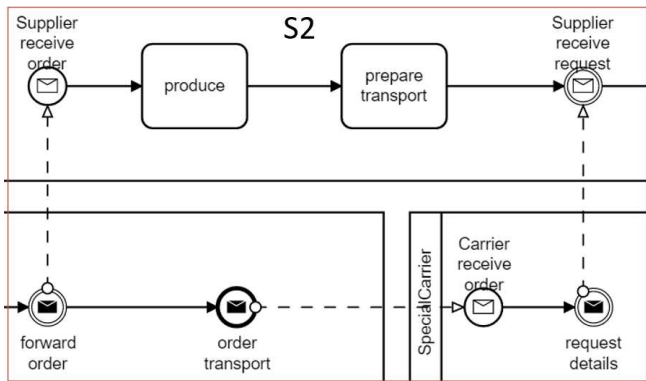


Fig. 6. Subgraph S2 of the BPMN model shown in Fig. 3.

Closest to our work is research on creation of smart contracts from BPMN model. Weber et al. [17] propose

architecture for creating smart contracts from a distributed application specified as a BPMN model. The transformation process has two phases, design and execution. In the design phase, the BPMN model is analysed and transformed into a distributed application that uses a blockchain smart contract, while in the second phase, the smart contract execution is supported. There are two components to the system, an off-chain component that accepts input from the external application and transforms it into calls to a smart contract method, and the on-chain component with smart contract methods that also includes a run-time monitor/mediator that manages the choreography of task activities represented by BPMN elements. From this work there were two significant follow up projects, Caterpillar [18] and Lorikeet [19].

Caterpillar has been further augmented with the ability to bind actors to roles dynamically using a proposed policy specification language based on Petri net semantics [23], [24]. The proposal has been incorporated into the Caterpillar project software and the approach was evaluated with positive results. Ciccio et al. [25] also support model-driven engineering using BPMN for generation of the methods of smart contracts and for demonstration of feasibility they use the Caterpillar [18] and Lorikeet [19] projects.

Loukil et al. [26] also start with a BPMN model, but one of their requirements is to address the issue of a smart contract immutability and hence requiring creation and deployment of a new contract any time new updates are required. Thus, instead of generating a compiled version of a smart contract from a BPMN model, they advocate creation and deployment of one generic smart contract that invokes predefined functions. They propose a system architecture that has three layers: Conceptual, Data, and Flow layer. The conceptual layer performs a set of activities that invoke functions of the Date layer.

Lu et al. [27] used business process modeling to support fungible and non-fungible asset management. In addition to BPMN modeling, their architecture also includes asset registry modeling for each fungible and non-fungible asset. For evaluation they use Lorikeet for BPMN modeling of selected use cases from government and industry, wherein the use cases include fungible/non-fungible asset registration, escrow for conditional payment, and asset swap.

VI. CONCLUSIONS AND FUTURE WORK

The significance of automated transformation of a BPMN model into methods of a smart contract is that it eases greatly the complexity of designing and developing smart contracts. Instead of developing a smart contract from specification, the application is modeled in BPMN and then transformed into methods of a smart contract. Our approach goes a step further in that it also automatically deploys the smart contract(s) on a blockchain. For deployment, the blockchain infrastructure needs to be available and the TABS tool needs to be configured with methods to connect to a blockchain. However, once the tool is supplied with configuration information on how to connect to blockchains, it can be used by a BPMN modeler without expertise to create applications that are based on blockchain smart contract.

Once a BPMN model is prepared, the modeler interacts with the TABS tool by guiding the transformation by using the tool's tabs (buttons) and selecting appropriate patterns for processing off-chain, either to support collaborative transaction or to reduce the cost of mainchain processing by

moving computation to a sidechain. In addition, a developer also needs to provide code for each of the individual tasks. However, each task is limited to its own scope with input and output parameters together with blockchain data being the only information that the task shares with the rest of the system.

We are working on removing some of the assumptions we made on BPMN model. In particular, we are working on supporting BPMN modelling that does include loops, albeit starting with simple static loops, and sub-processes that have loops or are concurrent.

Another issue to overcome is the lack of BPMN expressiveness in describing data that can flow along sequence flows that describe the control flow. We use BPMN data associations for that purpose, but they are optional and serve as textual descriptions only. They lack representation of data properties, such as types, multiplicity of values, or constraints on values.

Although much work needs to be done before a BPMN modeler can avail herself of tools to develop distributed applications on smart contracts, research progress is bringing smart contracts closer to being the truly disruptive force that some claim it to be.

REFERENCES

- [1] D. Yang, C. Long, H. Xu, S. Peng, 2020. A Review on Scalability of Blockchain. In *Proceedings of the 2020 The 2nd International Conference on Blockchain Technology (ICBCT'20)*. Association for Computing Machinery, New York, NY, USA, 1–6. DOI: <https://doi.org/10.1145/3390566.3391665>
- [2] Shard Chains | Ethereum.org. <https://ethereum.org/en/eth2/shard-chains/>, 2021 (access 26 March 2022).
- [3] P. J. Taylor, T. Dargahi, Dehghantaha, R. M. Parizi, 2019. A Systematic Literature Review Of Blockchain Cyber Security - ScienceDirect. A systematic literature review of blockchain cyber security - ScienceDirect. <https://www.sciencedirect.com/science/article/pii/S2352864818301536>.
- [4] S. Khan, F. Loukil, C. Ghedira-Guegan, E. Benkhalifa, A. Bani-Hani, 2021. Blockchain smart contracts: Applications, challenges, and future trends. *Peer Peer Netw Appl*. 2021 Apr 18:1-25. doi: 10.1007/s12083-021-01127-0. Epub ahead of print. PMID: 33897937; PMCID: PMC8053233.
- [5] A. Vacca, A. Annalisa et al. 2021. A systematic literature review of blockchain and smart contract development: Techniques, tools, and open challenges. *J. Syst. Softw.* 174 (2021): 110891.
- [6] R. Belchior, A. Vasconcelos, S. Guerreiro, M. Correia. 2021. A Survey on Blockchain Interoperability: Past, Present, and Future Trends. *ACM Comput. Surv.* 54, 8, Article 168, 41 pages. DOI: <https://doi.org/10.1145/3471140>
- [7] K. Saito, H. Yamada, 2016. What's So Different about Blockchain? Blockchain: Probabilistic State Machine. 2016 ICDCSW. 168-175.
- [8] J. A. Garcia-Garcia, N. Sánchez-Gómez, D. Lizcano, M. J. Escalona and T. Wojdyński, "Using Blockchain to Improve Collaborative Business Process Management: Systematic Literature Review," in *IEEE Access*, vol. 8, pp. 142312-142336, 2020, doi: 10.1109/ACCESS.2020.3013911.
- [9] C. Lauster, P. Klinger, N. Schwab, F. Bodendorf, 2020. Literature Review Linking Blockchain and Business Process Management in 15th International Conference on Wirtschaftsinformatik, March 08-11, 2020, Potsdam, Germany. https://doi.org/10.30844/wi_2020_r10-klinger
- [10] O. Levasseur, M. Iqbal, R. Matulevičius, 2021. Survey of Model-Driven Engineering Techniques for Blockchain-Based Applications. PoEM'21 Forum: 14th IFIP WG 8.1 Working Conference on the Practice of Enterprise Modelling,
- [11] P. Tolmach, Y. Li, S. Lin, Y. Liu, Z. Li. 2021. A Survey of Smart Contract Formal Specification and Verification. *ACM Comput. Surv.* 54, 7, Article 148 (September 2022), 38 pages. DOI: <https://doi.org/10.1145/3464421>
- [12] L. Dikmans, 2008. Transforming BPMN into BPEL: Why and How. <https://www.oracle.com/technical-resources/articles/dikmans-bpm.html> (access 26 March 2022).
- [13] [41] P. Bodorik, C. Liu, D. Jutla, 2021. Using FSMs to Find Patterns for Off-Chain Computing: Finding Patterns for Off-Chain Computing with FSMs. In 2021 The 3rd International Conference on Blockchain Technology (ICBCT '21), March 26–28, 2021, Shanghai, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3460537.3460565>
- [14] [42] C. Liu, P. Bodorik, D. Jutla, 2021. A Tool for Moving Blockchain Computations Off-Chain. In Proceedings of 2021 3rd ACM International Symposium on Blockchain and Secure Critical Infrastructure (BSCI'21), June 3–7, 2021, Virtual Event, Hong Kong. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3457337.3457848>
- [15] G. Liu, P. Bodorik, D. Jutla. 2021, From BPMN to smart contracts on blockchains: Transforming BPMN to DE-HSM multi-modal model, 7th IEEE International Conference on Engineering and Emerging Technologies (ICEET), 2021, pp. 1-7, doi: 10.1109/ICEET53442.2021.9659771.
- [16] P. Bodorik, C. G. Liu, D. Jutla. 2022. TABS: Transforming Automatically BPMN Models into Smart Contracts. *Blockchain: Research and Applications*, under review. <https://web.cs.dal.ca/~gang/BCRA/>
- [17] I. Weber, X. Xu, R. Riveret, G. Governatori, A. Ponomarev, J. Mendling, 2016. Untrusted Business Process Monitoring and Execution Using Blockchain. *Proceedings of Business Process Management Conference*, Rome, Italy, 2016, 329-347.
- [18] O. López-Pintado, L. García-Bañuelos, M. Dumas, I. Weber, A. Ponomarev, 2019. Caterpillar: a business process execution engine on the Ethereum blockchain. *Softw: Pract Exper*. 2019; 49: 1162– 1193. <https://doi.org/10.1002/spe.2702>
- [19] A. B. Tran, Q. Lu, I. Weber, 2018. Lorikeet: A Model-Driven Engineering Tool for Blockchain-Based Business Process Execution and Asset Management. 16th International Conference on Business Process Management, BPM 2018, Sydney, Australia, September 9–14, 2018, pp. 56–60.
- [20] Business Process Model And Notation. Object Management Group. <https://www.omg.org/spec/BPMN/2.0/> (access 26 March 2022).
- [21] A. Girault and B. Lee and E.A. Lee (1999). Hierarchical Finite State Machines with Multiple Concurrency Models. *IEEE Transactions on Computer-Aided Design*, June 6, 1999. 742--760
- [22] W. Fdhila, S. Rinderle-Ma, D. Knaplesch, and M. Reichert. Change and compliance in collaborative processes. In *IEEE SCC*, pages 162{169, 2015.
- [23] O. López-Pintado, M. Dumas, L. García-Bañuelos, I. Weber, 2019. Dynamic role binding in blockchain-based collaborative business processes, in: *Advanced Information Systems Engineering - 31st International Conference, CAISE 2019, Rome, Italy, June 3–7, 2019, Proceedings*, 2019, pp. 399–414.
- [24] O. López-Pintado, M. Dumas, L. García-Bañuelos, I. Weber, 2022. Controlled flexibility in blockchain-based collaborative business processes. *Inf. Syst.*, 104, 101622.
- [25] C. Di Ciccio, A. Cecconi, M. Dumas et al., 2019. Blockchain Support for Collaborative Business Processes. *Informatik Spektrum* 42, 182–190 (2019). <https://doi.org/10.1007/s00287-019-01178-x>
- [26] F. Loukil, K. Boukadi, M. Abed, C. Ghedira-Guegan, 2021. Decentralized Collaborative Business Process Execution Using Blockchain. *World Wide Web*, Springer Verlag, 2021, (10.1007/s11280-021-00901-7). (hal-03250846)
- [27] Q. Lu, A. B. Tran, I. Weber, et al. Integrated model-driven engineering of blockchain applications for business processes and asset management. *Softw Pract Exper*. 2021; 51: 1059– 1079. <https://doi.org/10.1002/spe.2931>